



Doric System DLL

User Manual

Version 1.1.0

Contents

1 Overview	3
1.1 Description	3
1.2 System requirements	3
2 Installation	4
2.1 Installation	4
3 Doric System	6
3.1 Namespace	6
3.2 Enumerations	6
3.3 Structures	6
3.4 Functions	7
3.5 Examples	9
4 OTPG	10
4.1 Namespace	10
4.2 Enumerations	10
4.3 Structures	10
4.4 Functions	11
4.5 Examples	13
5 Light Source	17
5.1 Namespace	17
5.2 Enumerations	17
5.3 Structures	17
5.4 Functions	19
5.5 Examples	21
6 Rotary Joint	25
6.1 Namespace	25
6.2 Enumerations	25
6.3 Structures	26
6.4 Functions	26
6.5 Examples	28
7 License(s)	32
8 Support	33
8.1 Contact us	33

Overview

1.1 Description

This is a guide to help the users in building their own software application using Doric System DLL(s) libraries. The DLL(s) are compiled in a **64 bits architecture** using MSVC 2019 compilers and were tested using different programming language such as C++, Python, Matlab/Octave etc.

The Doric System DLL(s) comes prepackaged with the DLL(s) necessary to the application and with some example scripts. Please ensure that your application can ***link to all DLL(s) files supplied***.

1.2 System requirements

Windows compatible only

- Operating system: Microsoft Windows 10, 11; 64-bit
- Processor speed: 2 GHz and 4 cores minimum, 3 GHz and 8 cores recommended
- Memory: 4 GB RAM minimum, 8 GB RAM recommended
- Hard drive: 1 GB of free hard disk space, SSD recommended

Installation

2.1 Installation

2.1.1 C++ (Qt)

For usage in a Qt environment, make sure to follow instructions :

1. Add library (DoricSystem.lib) to your project. Make sure the .dll (DoricSystem.dll) is in the same folder/directory as the library.
 - Right-click on your project then select : Add Library...
 - Select External library and then click Next>
 - Browse for the library file (should be in **C:PATH_TO_LIB/DoricSystemDLL/API/lib/x64/release**)
 - Browse for the include files (should be in **C:PATH_TO_LIB/DoricSystemDLL/API/include**)
 - Make sure to select [*Library inside "debug" or "release" subfolders*] option to be able to build in release and debug mode.
2. Add all the Qt .dll(s) in the same directory as the main library or make sure that your coding environment can have access to them.
 - This step can be achieve in multiples ways, it really depends on the users habits.
 - Make sure to select [*Library inside "debug" or "release" subfolders*] option to be able to build in release and debug mode.
3. Add `#include doric_system_wrapper.h` to your project to access all the enumerations, structures, functions etc...

2.1.2 C++ (Visual Studio)

For usage in a Visual Studio environment, make sure to follow instructions :

1. Add library (DoricSystem.lib) to your project. Make sure the .dll (DoricSystem.dll) is in the same folder/directory as the library.
 - Right-click on your project then select : Add ->Existing Item
 - Select the DoricSystem.lib to be added to the project
 - Make sure to select [*Library inside "debug" or "release" subfolders*] option to be able to build in release and debug mode.
2. Add all the Qt .dll(s) in the same directory as the main library or make sure that your coding environment can have access to them.
 - This step can be achieve in multiples ways, it really depends on the users habits.

- Make sure to select [*Library inside "debug" or "release" subfolders*] option to be able to build in release and debug mode.
3. Make sure your project can link to all headers (include) files (.h)
 - Again, this step can be achieve in multiples ways, it really depends on the users habits.
 4. Add `#include doric_system_wrapper.h` to your project to access all the enumerations, structures, functions etc...
 5. There's also some small example for different device. They can be found int the different folders :
 - `C:/PATH_TO_LIBRARY/DoricSystemDLL/Examples/CPP/OTPG.`
 - `C:/PATH_TO_LIBRARY/DoricSystemDLL/Examples/CPP/LightSource.`
 - `C:/PATH_TO_LIBRARY/DoricSystemDLL/Examples/CPP/RotaryJoint.`

2.1.3 Python (Script)

For usage in a Python environment, make sure to follow instructions :

1. Make sure your have any Python environment already installed on your computer.
2. Add library (DoricSystem.lib) to your project. Make sure the .dll (DoricSystem.dll) is in the same folder/directory as the library and your Python script(s).
 - To do this step, you only need to copy the .lib and .dll files.
3. Add all the Qt .dll(s) in the same directory as the main library or make sure that your Python environment can have access to them.
 - To do this step, you only need to copy the all files included in the `C:/PATH_TO_LIBRARY/DoricSystemDLL/API/lib/x64/release/Qt` folder.
 - Else you might need to copy the all files included in the `C:/PATH_TO_LIBRARY/DoricSystemDLL/API/lib/x64/debug/Qt` folder if your planning to use debugging environment.
4. Add `from lightsource_defs import*` to your project to access all the enumerations and structures for the light source(s).
5. Add `from otpg_defs import*` to your project to access all the enumerations and structures for the OTPG(s).
6. There's also many pre-created classes as examples. They can be found int the different folders :
 - `C:/PATH_TO_LIBRARY/DoricSystemDLL/Examples/Python/OTPG/otpg_example_classes.`
 - `C:/PATH_TO_LIBRARY/DoricSystemDLL/Examples/Python/LightSource/lightsource_example_classes.`
 - `C:/PATH_TO_LIBRARY/DoricSystemDLL/Examples/Python/RotaryJoint/rotaryjoint_example_classes.`

2.1.4 Matlab/Octave (Script)

In progress

Doric System

3.1 Namespace

```
namespace Doric::System {}
```

3.2 Enumerations

```
namespace Doric {  
    namespace System {  
  
        typedef enum class TriggerType{  
            Triggered,  
            Gated,  
            Manual = 255  
        }TriggerType;  
  
        typedef enum class TriggerMode{  
            Uninterrupted,  
            Pause,  
            Continue,  
            Restart  
        }TriggerMode;  
  
        typedef enum class Channel{  
            Channel_1,  
            Channel_2,  
            Channel_3,  
            Channel_4,  
            Channel_5,  
            Channel_6,  
            Channel_7,  
            Channel_8,  
            Undefined = 255  
        }Channel;  
    }  
}
```

3.3 Structures

```
namespace Doric {  
    namespace System {
```

```

struct TimeSeriesProperties{
    uint32_t activeTimeMs = 1000
    (Minimum = 1000, Maximum = 4,294,967,295)
    uint16_t numberOfSeries = 1
    (Minimum = 0, Maximum = 65,536)
    uint32_t delayBetweenSeriesMs = 1000
    (Minimum = 1000, Maximum = 4,294,967,295)
    bool isUsingTimeSeries = False
    (True = Device uses Time Series, False = Device doesn't used Time Series)
}
}
}

```

3.4 Functions

3.4.1 List of functions

```

namespace Doric {
    namespace System {
        void init(bool debuggerActive);
        void quit();
        void wait(int delayInMSec);
        void available_devices();
        void available_devices_with_ports();
        void open_device(int portNumber);
        void close_device(int portNumber);
    }
}

```

3.4.2 Functions declaration and definition

The following tables indicates the [DORIC SYSTEM] functions available with their inputs/outputs arguments

Function	Input(s)	Output(s)
init	bool debuggerActive TRUE = ACTIVED FALSE = DEACTIVATED	void

This function is used to initialize the library. This function must be called first to activate the Doric device(s) detection and more.

Function	Input(s)	Output(s)
quit	void	void

This function is used to quit the library. This function must be called at the end to deactivate the Doric device(s) detection and more.

Function	Input(s)	Output(s)
<code>wait</code>	<code>int delayInMSec</code> Value in milliseconds. (Max = 65535)	<code>void</code>

This function is used to wait a certain amount of time in the library. This function is usually called to let the time to the library to execute its different functionalities.

Function	Input(s)	Output(s)
<code>available_devices</code>	<code>void</code>	<code>void</code>

This function is used to get a list of all the Doric devices connected to the user computer. This function will only print a debug message with the name of all available devices detected.

Function	Input(s)	Output(s)
<code>available_devices_with_ports</code>	<code>void</code>	<code>void</code>

This function is used to get a list of all the Doric devices connected to the user computer. This function will only print a debug message with the name and ports of all available devices detected.

Function	Input(s)	Output(s)
<code>open_device</code>	<code>int portNumber</code> USB-port number of the device.	<code>void</code>

This function is used to connect to a Doric devices. The function need the USB port number of the device to be connected.

Function	Input(s)	Output(s)
<code>close_device</code>	<code>int portNumber</code> USB-port number of the device.	<code>void</code>

This function is used to disconnect to a Doric devices. The function need the USB port number of the device to

be disconnected.

3.5 Examples

Example(s) of the use of the Doric System library can be found in each of the others examples since it is required to be initialized before using any Doric devices.

- [OTPG Examples](#)
- [Light Source\(s\) Examples](#)
- [Rotary Joint Examples](#)

OTPG

4.1 Namespace

```
namespace Doric::OTPG {}
```

4.2 Enumerations

```
namespace Doric {  
    namespace OTPG {  
  
        typedef enum class Mode{  
            Off,  
            Square,  
            Input  
        }Mode;  
  
        typedef enum class SamplingFrequency{  
            Freq_10Hz,  
            Freq_100Hz,  
            Freq_500Hz,  
            Freq_1kHz,  
            Freq_5kHz,  
            Freq_10kHz  
        }SamplingFrequency;  
    }  
}
```

4.3 Structures

```
namespace Doric {  
    namespace OTPG {  
  
        struct Settings {  
  
            Doric::System::Channel channelId = Doric::System::Channel::Channel_1  
            (see Doric::System::Channel)  
            Doric::OTPG::Mode mode = Doric::OTPG::Mode::Off  
            (see Doric::OTPG::Mode)  
            Doric::System::Channel triggerSource = Doric::System::Channel::Undefined  
            (see Doric::System::Channel)  
            Doric::System::TriggerType triggerType = Doric::System::TriggerType::Manual  
            (see Doric::System::TriggerType)  
        }  
    }  
}
```

```

Doric::System::TriggerMode triggerMode = Doric::System::TriggerType::Uninterrupted
(see Doric::System::TriggerMode)
uint32_t startingDelayMs = 0
(Minimum = 0, Maximum = 4,294,967,296)
uint32_t delayBetweenSeqMs = 0
(Minimum = 0, Maximum = 4,294,967,295)
double periodMs = 10
(Minimum = 0, Maximum = 4,294,967,296)
double timeOnMs = 0
(Minimum = 0, Maximum = 4,294,967,296)
uint16_t nbOfSeq = 1
(Minimum = 1, Maximum = 65,536)
uint16_t nbOfPulsesPerSeq = 0
(Minimum = 0, Maximum = 65,536)
bool isRepeatableSequence = False
(True = sequence(s) repeated, False = sequence(s) not repeated)
bool isInverted = False
(True = sequence(s) inverted, False = sequence(s) not inverted)
}

struct SamplingParameters {

    Doric::System::Channel triggerSource = Doric::System::Channel::Undefined
(see Doric::System::Channel)
    Doric::OTPG::SamplingFrequency samplingFrequency = Doric::OTPG::SamplingFrequency::Freq_1kHz
(see Doric::OTPG::SamplingFrequency)

}
}
}

```

4.4 Functions

4.4.1 List of functions

```

namespace Doric {
    namespace OTPG {
        void otpg_start_all(int portNumber);
        void otpg_stop_all(int portNumber);
        void otpg_send_settings(int portNumber, Doric::OTPG::Settings settings);
        void otpg_send_delete_settings(int portNumber, Doric::System::Channel channelIndex);
        void otpg_send_sampling_parameters(int portNumber,
                                           Doric::OTPG::SamplingParameters samplingParameters);
        void otpg_send_timeseries_properties(int portNumber,
                                           Doric::System::TimeSeriesProperties timeseriesProperties);
    }
}

```

4.4.2 Functions declaration and definition

The following tables indicates the [OTPG] functions available with their inputs/outputs arguments

Function	Input(s)	Output(s)
<code>otpg_start_all</code>	<code>int portNumber</code> USB-port number of the device.	<code>void</code>

This function is used to start an OTPG. All channel(s) previously set will start its sequence(s) when this function is called.

Function	Input(s)	Output(s)
<code>otpg_stop_all</code>	<code>int portNumber</code> USB-port number of the device.	<code>void</code>

This function is used to stop an OTPG. All channel(s) previously set will stop it's sequence(s) when this function is called.

Function	Input(s)	Output(s)
<code>otpg_send_settings</code>	<code>int portNumber</code> USB-port number of the device. <code>Doric::OTPG::Settings settings</code> Structure of a OTPG settings	<code>void</code>

This function is used to send an OTPG settings to one of the channel(s) of the driver. This will configure the channel with the structure sent.

Function	Input(s)	Output(s)
<code>otpg_send_delete_settings</code>	<code>int portNumber</code> USB-port number of the device. <code>int channelIndex</code> Channel to be deleted	<code>void</code>

This function is used to delete an OTPG settings from a channel. This will invalidate the channel so it wont be active anymore.

Function	Input(s)	Output(s)
<code>otpg_send_sampling_parameters</code>	int <code>portNumber</code> USB-port number of the device. Doric::OTPG::SamplingParameters samplingParameter Structure of a OTPG sampling parameters	void

This function is used to send an OTPG sampling parameters. Frequency of the sampling/acquisition and the triggering of the OTPG driver can be control with those parameters contained in the structure.

Function	Input(s)	Output(s)
<code>otpg_send_timeseries_properties</code>	int <code>portNumber</code> USB-port number of the device. Doric::System::TimeSeriesProperties timeseriesProperties Structure of a Doric Time Series properties	void

This function is used to send an Doric Time Series properties. The Doric Time Series properties let you customize the moment input/outputs of the drivers are active when an acquisition is started. It let you choose a number of series, an active time and the delay between each of those series.

4.5 Examples

4.5.1 Python

```

from ctypes import*

from otpg_example_classes import*

import sys
import os

# OTPG USB port #
OTPG_USB_PORT_NUMBER = 10

mDLL = cdll.LoadLibrary('C:/PATH_TO_THE_DORIC_LIB/DoricSystem.dll')

def main():

    # Initialize communication with Doric devices #
    mDLL.init(True)

    # Wait for initialization to complete #
    mDLL.wait(3000)

    # List available(s) device(s) #

```

```

mDLL.available_devices_with_ports()

# Open a device. In this case, a OTPG is connected on port 10 #
mDLL.open_device(OTPG_USB_PORT_NUMBER)

# Wait for the device to be initialized #
mDLL.wait(3000)

#####
##### LIST OF MULTIPLE DIFFERENT EXAMPLE OF OTPG USAGE #####
#####

# # Example_OTPG_CW(mDLL, OTPG_USB_PORT_NUMBER, Channel.Channel_1)
# # Example_OTPG_Square(mDLL, OTPG_USB_PORT_NUMBER, Channel.Channel_1)
# # Example_OTPG_Triggered(mDLL, OTPG_USB_PORT_NUMBER, Channel.Channel_1, Channel.Channel_2)
# # Example_OTPG_Gated(mDLL, OTPG_USB_PORT_NUMBER, Channel.Channel_1, Channel.Channel_2)
#
# # Example_OTPG_Square(mDLL, OTPG_USB_PORT_NUMBER, Channel.Channel_1)
# # Example_OTPG_CW(mDLL, OTPG_USB_PORT_NUMBER, Channel.Channel_2)
# #
# # Example_OTPGSamplingParameters_Default(mDLL, OTPG_USB_PORT_NUMBER)
# # Example_OTPGSamplingParameters_Triggered(mDLL, OTPG_USB_PORT_NUMBER)
# #
# # Example_OTPG_TimeSeries_Default(mDLL, OTPG_USB_PORT_NUMBER)
# # Example_OTPG_TimeSeries_Active(mDLL, OTPG_USB_PORT_NUMBER)

# Simply create a square signal (see otpg_example_classes.py for more information or examples) #
Example_OTPG_Square(mDLL, OTPG_USB_PORT_NUMBER, Channel.Channel_1)

# Wait for the settings to be sent to the device #
mDLL.wait(1000)

# Start the OTPG (connected on port#10) #
mDLL.otpg_start_all(OTPG_USB_PORT_NUMBER)

# Let it run 10 sec #
mDLL.wait(10000)

# Stop the OTPG (connected on port#10) #
mDLL.otpg_stop_all(OTPG_USB_PORT_NUMBER)

# Close the OTPG #
mDLL.close_device(OTPG_USB_PORT_NUMBER)

# Wait for device to close #
mDLL.wait(1000)

# Quit the library #
mDLL.quit()

try:
    sys.exit(0)
except SystemExit:
    os._exit(0)

while (True): continue

```

```

if __name__ == '__main__':
    try:
        main()
    except KeyboardInterrupt:
        print('Interrupted (keyboard)')

        # Stop the OTPG (connected on port#10) #
        mDLL.otpg_stop_all(OTPG_USB_PORT_NUMBER)

        # Close the OTPG #
        mDLL.close_device(OTPG_USB_PORT_NUMBER)

        # Wait for device to close #
        mDLL.wait(1000)

        # Quit the library #
        mDLL.quit()

    try:
        sys.exit(0)
    except SystemExit:
        os._exit(0)

```

4.5.2 C++

```

#include <iostream>

#include "doric_system_wrapper.h"

int main()
{
    /* Create an OTPG settings*/
    Doric::OTPG::Settings* otpg_settings = new Doric::OTPG::Settings();

    /* Configure a square mode on channel #1 of OTPG */
    otpg_settings->channelIdx = Doric::System::Channel::Channel_1;
    otpg_settings->mode = Doric::OTPG::Mode::Square;

    /* Configure a manual triggering (start with OTPG start) */
    otpg_settings->triggerMode = Doric::System::TriggerMode::Uninterrupted;
    otpg_settings->triggerType = Doric::System::TriggerType::Manual;
    otpg_settings->triggerSource = Doric::System::Channel::Undefined;

    /* Sequence(s) will be repeated once done & trigger received */
    otpg_settings->isRepeatableSequence = false;
    otpg_settings->isInverted = false;

    /* Initialize communication with Doric devices */
    Doric::System::init(true);

    /* Wait for initialization to complete*/
    Doric::System::wait(5000);

    /* List available(s) device(s) */
    Doric::System::available_devices_with_ports();
}

```

```

/* OTPG USB port */
int OTPG_port = 10;

/* Open a device. In this case, a OTPG is connected on port 10 */
Doric::System::open_device(OTPG_port);

/* Wait for the device to be initialized */
Doric::System::wait(7000);

/* Send the settings created previously to device on port #10 */
Doric::OTPG::otpg_send_settings(OTPG_port, otpg_settings);

/* Wait for the settings to be initialize */
Doric::System::wait(1000);

/* Start the OTPG (connected on port#10) */
Doric::OTPG::otpg_start_all(OTPG_port);

/* Let it run 10 sec */
Doric::System::wait(10000);

/* Stop the OTPG (connected on port#10) */
Doric::OTPG::otpg_stop_all(OTPG_port);

/* Close the OTPG */
Doric::System::close_device(OTPG_port);

/* Wait for device to close */
Doric::System::wait(1000);

Doric::System::quit();
}

```

4.5.3 Matlab/Octave

In progress (Matlab code here)

Light Source

5.1 Namespace

```
namespace Doric::LightSource {}
```

5.2 Enumerations

```
namespace Doric {
    namespace LightSource {

        typedef enum class Mode{
            Off,
            CW,
            ExtTTL,
            ExtAnalog,
            Square,
            Complex,
            Custom
        }Mode;

        typedef enum class CurrentMode{
            Normal,
            LowPower,
            Overdrive
        }CurrentMode;
    }
}
```

5.3 Structures

```
namespace Doric {
    namespace LightSource {

        struct Settings {

            Doric::System::Channel channelId = Doric::System::Channel::Channel_1
            (see Doric::System::Channel)
            Doric::LightSource::Mode mode = Doric::LightSource::Mode::Off
            (see Doric::LightSource::Mode)
            bool isTTLOutput = False
            (True = Output TTL, False = Output "ADC")
            Doric::System::TriggerType triggerType = Doric::System::TriggerType::Manual
        }
    }
}
```

```

    (see Doric::System::TriggerType)
    Doric::System::TriggerMode triggerMode = Doric::System::TriggerMode::Uninterrupted
    (see Doric::System::TriggerMode)
    bool isTriggerRepeatable = False
    (True = trigger repeated, False = trigger not repeated)
    Doric::LightSource::CurrentMode currentMode = Doric::LightSource::CurrentMode::Normal
    (Doric::LightSource::CurrentMode)
    uint16_t[1000] customDataPoints
    (Vector of 1000 uint16_t value initialize with [0])
    Doric::LightSource::TTLModulation ttlModulation
    (see Doric::LightSource::TTLModulation)
    uint8_t nbComplexModulations = 0
    (Minimum = 0, Maximum = 32)
    Doric::LightSource::ComplexModulation complexModulations
    (see Doric::LightSource::ComplexModulation)
}

struct TTLModulation {
    uint16_t currentmA = 0
    (Minimum = 0, Maximum = depend on light source, usually 2000)
    uint32_t startingDelayMs = 0
    (Minimum = 0, Maximum = 4,294,967,295)
    uint32_t delayBetweenSeqMs = 0
    (Minimum = 0, Maximum = 4,294,967,295)
    double periodMs = 100
    (Minimum = 0, Maximum = 4,294,967,296)
    double timeOnMs = 50
    (Minimum = 0, Maximum = 4,294,967,296)
    uint16_t risingTimeMs = 0
    (Minimum = 0, Maximum = 65,535)
    uint16_t fallingTimeMs = 0
    (Minimum = 0, Maximum = 65,535)
    uint16_t nbOfSeq = 1
    (Minimum = 1, Maximum = 65,535)
    uint16_t nbOfPulsesPerSeq = 0
    (Minimum = 0, Maximum = 65,535)
}

struct ComplexModulation {
    Doric::LightSource::Mode mode = Doric::LightSource::Mode::CW
    (see Doric::LightSource::Mode)
    uint16_t currentmA = 0
    (Minimum = 0, Maximum = 65,535)
    uint32_t delayBetweenSeqMs = 0
    (Minimum = 0, Maximum = 4,294,967,295)
    double periodMs = 100
    (Minimum = 0, Maximum = 4,294,967,296)
    double timeOnMs = 50
    (Minimum = 0, Maximum = 4,294,967,296)
    uint16_t nbOfSeq = 1
    (Minimum = 1, Maximum = 65,535)
    uint16_t nbOfPulsesPerSeq = 0
    (Minimum = 0, Maximum = 65,535)
    uint32_t startingDelayMs = 0
    (Minimum = 0, Maximum = 4,294,967,295)
}

```

```

    }
}

```

5.4 Functions

5.4.1 List of functions

```

namespace Doric {
    namespace LightSource {
        void ls_start_all(int portNumber);
        void ls_stop_all(int portNumber);
        void ls_start_channel(int portNumber, int channelIndex);
        void ls_stop_channel(int portNumber, int channelIndex);
        void ls_send_settings(int portNumber, LightSourceSettingsStruct settings);
        void ls_send_current(int portNumber, int channelIndex, uint16_t current);
    }
}

```

5.4.2 Functions declaration and definition

The following tables indicates the [Light Source] functions available with their inputs/outputs arguments

Function	Input(s)	Output(s)
ls_start_all	int portNumber	void
USB-port number of the device.		

This function is used to start all the light source driver channel(s). All channel(s) previously set will start its sequence(s) when this function is called.

Function	Input(s)	Output(s)
ls_stop_all	int portNumber	void
USB-port number of the device.		

This function is used to stop all the light source driver channel(s). All channel(s) previously set will start its sequence(s) when this function is called.

Function	Input(s)	Output(s)
ls_start_channel	int portNumber	void
USB-port number of the device.		
	int channelIndex	
Index of the channel ([0] based)		

This function is used to start a single light source driver channel. The channel previously set will start its sequence(s)

when this function is called.

Function	Input(s)	Output(s)
<code>ls_stop_channel</code>	<code>int portNumber</code> USB-port number of the device. <code>int channelIndex</code> Index of the channel ([0] based)	<code>void</code>

This function is used to stop a single light source driver channel. The channel previously set will stop its sequence(s) when this function is called.

Function	Input(s)	Output(s)
<code>ls_send_settings</code>	<code>int portNumber</code> USB-port number of the device. <code>Doric::LightSource::Settings settings;</code> Structure of a light source settings	<code>void</code>

This function is used to send a light source settings to one of the channel(s) of the driver. This will configure the channel with the structure sent.

Function	Input(s)	Output(s)
<code>ls_send_current</code>	<code>int portNumber</code> USB-port number of the device. <code>int channelIndex</code> Index of the channel ([0] based) <code>uint16_t current</code> Value of the current in mA	<code>void</code>

This function is used to modify the light source current of one of the channel(s) of the driver. This can be call at every moment, even when the light source sequence(s) is started.

5.5 Examples

5.5.1 Python

```
from ctypes import*

from lightsource_example_classes import*

import sys
import os

# LIGHT SOURCE USB port #
LIGHT_USB_PORT_NUMBER = 5

mDLL = cdll.LoadLibrary('C:/PATH_TO_THE_DORIC_LIB/DoricSystem.dll')

def main():

    # Initialize communication with Doric devices #
    mDLL.init(True)

    # Wait for initialization to complete #
    mDLL.wait(5000)

    # List available(s) device(s) #
    mDLL.available_devices_with_ports()

    # Open a device. In this case, a laser is connected on port 10 #
    mDLL.open_device(LIGHT_USB_PORT_NUMBER)

    # Wait for the device to be initialized #
    mDLL.wait(5000)

    #####
    ##### LIST OF MULTIPLE DIFFERENT EXAMPLE OF LIGHT USAGE #####
    #####

    # # Example_Light_CW(mDLL, LIGHT_USB_PORT_NUMBER, Channel.Channel_1)
    # # Example_Light_ExtTTL(mDLL, LIGHT_USB_PORT_NUMBER, Channel.Channel_1)
    # # Example_Light_ExtAnalog(mDLL, LIGHT_USB_PORT_NUMBER, Channel.Channel_1)
    # # Example_Light_Triggered(mDLL, LIGHT_USB_PORT_NUMBER, Channel.Channel_1)
    #
    # # Example_Light_Gated(mDLL, LIGHT_USB_PORT_NUMBER, Channel.Channel_1)
    # # Example_Light_Complex(mDLL, LIGHT_USB_PORT_NUMBER, Channel.Channel_1)
    # #
    # # Example_Light_Custom(mDLL, LIGHT_USB_PORT_NUMBER, Channel.Channel_1)

    # Simply create a square signal (see lightsource_example_classes.py for more information or examples) :
    Example_Light_Square(mDLL, LIGHT_USB_PORT_NUMBER, Channel.Channel_1)

    # Wait for the settings to be sent to the device #
    mDLL.wait(1000)

    # Start the laser (all channels) (connected on port#5) #
    mDLL.ls_start_all(LIGHT_USB_PORT_NUMBER)

    # Let it run 10 sec #
```

```

mDLL.wait(10000)

# Stop the laser (all channels) (connected on port#5) #
mDLL.ls_stop_all(LIGHT_USB_PORT_NUMBER)

# Close the laser #
mDLL.close_device(LIGHT_USB_PORT_NUMBER)

# Wait for device to close #
mDLL.wait(1000)

# Quit the library #
mDLL.quit()

try:
    sys.exit(0)
except SystemExit:
    os._exit(0)

while (True): continue

if __name__ == '__main__':
    try:
        main()
    except KeyboardInterrupt:
        print('Interrupted (keyboard)')

        # Stop the laser (all channels) (connected on port#5) #
        mDLL.ls_stop_all(LIGHT_USB_PORT_NUMBER)

        # Close the laser #
        mDLL.close_device(LIGHT_USB_PORT_NUMBER)

        # Wait for device to close #
        mDLL.wait(1000)

        # Quit the library #
        mDLL.quit()

        try:
            sys.exit(0)
        except SystemExit:
            os._exit(0)

```

5.5.2 C++

```

#include <iostream>

/* Include for DoricSystem library */
#include "doric_system_wrapper.h"

int main()
{
    /* Basic infinite square modulation */

```

```

Doric::LightSource::TTLModulation ttlModulation;

/* 55mA current */
ttlModulation.current = 55;

/* 50% duty cycle with 1 second period */
ttlModulation.timeOnMs = 500;
ttlModulation.periodMs = 1000;

/* Square mode, normal current, on channel#1 */
Doric::LightSource::Settings* lightsource_settings = new Doric::LightSource::Settings();
lightsource_settings->channelIdx = Doric::System::Channel::Channel_1;
lightsource_settings->mode = Doric::LightSource::Mode::Square;
lightsource_settings->currentMode = Doric::LightSource::CurrentMode::Normal;

lightsource_settings->ttlModulation = ttlModulation;

/* Initialize communication with Doric devices */
Doric::System::init(true);

/* Wait for initialization to complete*/
Doric::System::wait(10000);

/* List available(s) device(s) */
Doric::System::available_devices_with_ports();

/* Laser USB port */
int LASER_port = 5;

/* Open a device. In this case, a laser is connected on port 5 */
Doric::System::open_device(LASER_port);

/* Wait for the device to be initialize */
Doric::System::wait(5000);

/* Send the settings created previously to device on port #5 */
Doric::LightSource::ls_send_settings(LASER_port, lightsource_settings);

/* Start the channel#1 of the laser (connected on port#5) */
Doric::LightSource::ls_start_channel(LASER_port, Doric::System::Channel::Channel_1);

/* Let it run 10 sec */
Doric::System::wait(10000);

/* Stop the channel#1 of the laser (connected on port#5) */
Doric::LightSource::ls_stop_channel(LASER_port, Doric::System::Channel::Channel_1);

/* Close the laser */
Doric::System::close_device(LASER_port);

/* Wait for device to close */
Doric::System::wait(1000);

/* Quit library */
Doric::System::quit();
}

```

5.5.3 Matlab/Octave

In progress (Matlab code here)

Rotary Joint

6.1 Namespace

```
namespace Doric::RotaryJoint {}
```

6.2 Enumerations

```
namespace Doric {  
    namespace RotaryJoint {  
        typedef enum class SamplingRate{  
            kFreq_10Hz,  
            kFreq_30Hz,  
            kFreq_60Hz,  
            kFreq_120Hz,  
            kFreq_300Hz,  
            kFreq_600Hz,  
            kFreq_1200Hz,  
        }SamplingRate;  
  
        typedef enum class MotorSpeedFactor{  
            kSPEED_FACTOR_FULL = 1,  
            kSPEED_FACTOR_NORMAL = 2,  
            kSPEED_FACTOR_HALF = 4,  
        }MotorSpeedFactor;  
  
        typedef enum class MotorDirection{  
            kDirection_NoDirection,  
            kDirection_Clockwise ,  
            kDirection_CounterClockwise ,  
        }MotorDirection;  
  
        typedef enum class MotorMode{  
            kMode_Manual,  
            kMode_Continuous,  
            kMode_TurnPerSide,  
            kMode_Random  
        }MotorMode;  
    }  
}
```

6.3 Structures

```
namespace Doric {
    namespace RotaryJoint {

        struct Settings {

            RotaryJoint::SamplingRate samplingRate = RotaryJoint::SamplingRate::kFreq_1200Hz
            (see Doric::RotaryJoint::SamplingRate)
            RotaryJoint::MotorSpeedFactor motorSpeed = MotorSpeedFactor::kSPEED_FACTOR_NORMAL;
            (see Doric::RotaryJoint::MotorSpeedFactor)

        }

    }
}
```

6.4 Functions

6.4.1 List of functions

```
namespace Doric {
    namespace RotaryJoint {

        void rotary_joint_send_motor_power_on(int portNumber);
        void rotary_joint_send_motor_power_off(int portNumber);
        void rotary_joint_send_settings(int portNumber, Doric::RotaryJoint::Settings *settings);
        void rotary_joint_send_manual_control_activated(int portNumber);
        void rotary_joint_send_manual_control_deactivated(int portNumber);
        void rotary_joint_send_manual_control_duty_cycle(int portNumber, uint8_t dutyCycle);
        void rotary_joint_send_manual_control_direction(int portNumber,
            Doric::RotaryJoint::MotorDirection direction);
        void rotary_joint_send_manual_control_turn_per_side(int portNumber, uint8_t nbTurns);
        void rotary_joint_send_manual_control_mode(int portNumber, Doric::RotaryJoint::MotorMode mode);
    }
}
```

6.4.2 Functions declaration and definition

The following tables indicates the [Rotary Joint] functions available with their inputs/outputs arguments

Function	Input(s)	Output(s)
rotary_joint_send_motor_power_on	int portNumber	void
USB-port number of the device.		

This functions is used to power ON the motor.

Function	Input(s)	Output(s)
rotary_joint_send_motor_power_off	int portNumber	void
USB-port number of the device.		

This functions is used to power OFF the motor.

Function	Input(s)	Output(s)
rotary_joint_send_settings	<code>int portNumber</code> USB-port number of the device. <code>Doric::RotaryJoint::Settings</code> <code>*settings</code> Structure of a rotary joint settings	<code>void</code>

This functions is used to send the settings to the rotary joint.

Function	Input(s)	Output(s)
rotary_joint_send_manual_control_activated	<code>int portNumber</code> USB-port number of the device.	<code>void</code>

This functions is used to activate manual control of the rotary joint.

Function	Input(s)	Output(s)
rotary_joint_send_manual_control_deactivated	<code>int portNumber</code> USB-port number of the device.	<code>void</code>

This functions is used to deactivate manual control of the rotary joint.

Function	Input(s)	Output(s)
rotary_joint_send_manual_control_duty_cycle	<code>int portNumber</code> USB-port number of the device. <code>uint8 dutyCycle</code> Value of the duty cycle (0-100)	<code>void</code>

This functions is used to send the manual control duty cycle to the rotary joint.

Function	Input(s)	Output(s)
<code>rotary_joint_send_manual_control_direction</code>	<code>int portNumber</code> USB-port number of the device. <code>Doric::RotaryJoint::MotorDirection direction</code> Direction of the motor's rotation	<code>void</code>

This functions is used to send the manual control direction to the rotary joint.

Function	Input(s)	Output(s)
<code>rotary_joint_send_manual_control_turn_per_side</code>	<code>int portNumber</code> USB-port number of the device. <code>uint8_t nbTurns</code> Number of turns sent to the rotary joint (0-255)	<code>void</code>

This function is used to send the number of turns per side to the rotary joint.

Function	Input(s)	Output(s)
<code>rotary_joint_send_manual_control_mode</code>	<code>int portNumber</code> USB-port number of the device. <code>Doric::RotaryJoint::MotorMode mode</code> Mode of the motor	<code>void</code>

This function is used to send the motor mode to the rotary joint.

6.5 Examples

6.5.1 Python

```
from ctypes import*

from rotaryjoint_example_classes import *

import sys
import os

# Rotary Joint USB port #
ROTARY_JOINT_USB_PORT_NUMBER = 10
```

```

mDLL = cdll.LoadLibrary('C:/PATH_TO_THE_DORIC_LIB/DoricSystem.dll')

def main():

    # Initialize communication with Doric devices #
    mDLL.init(True)

    # Wait for initialization to complete #
    mDLL.wait(3000)

    # List available(s) device(s) #
    mDLL.available_devices_with_ports()

    # Open a device. In this case, a Rotary Joint is connected on port 10 #
    mDLL.open_device(ROTARY_JOINT_USB_PORT_NUMBER)

    # Wait for the device to be initialized #
    mDLL.wait(3000)

    #####
    ### LIST OF MULTIPLE DIFFERENT EXAMPLE OF ROTARY JOINT USAGE ###
    #####

    # Example_RotaryJoint_Default_Motor_ON_OFF(mDLL, ROTARY_JOINT_USB_PORT_NUMBER)
    # Example_RotaryJoint_Manual(mDLL, ROTARY_JOINT_USB_PORT_NUMBER)
    # Example_RotaryJoint_Manual_Continuous(mDLL, ROTARY_JOINT_USB_PORT_NUMBER)
    # Example_RotaryJoint_Manual_Random(mDLL, ROTARY_JOINT_USB_PORT_NUMBER)
    Example_RotaryJoint_Manual_TurnPerSide(mDLL, ROTARY_JOINT_USB_PORT_NUMBER)

    # Close the Rotary Joint #
    mDLL.close_device(ROTARY_JOINT_USB_PORT_NUMBER)

    # Wait for device to close #
    mDLL.wait(1000)

    # Quit the library #
    mDLL.quit()

    try:
        sys.exit(0)
    except SystemExit:
        os._exit(0)

    while (True): continue

if __name__ == '__main__':
    try:
        main()
    except KeyboardInterrupt:
        print('Interrupted (keyboard)')

        # Close the Rotary Joint #
        mDLL.close_device(ROTARY_JOINT_USB_PORT_NUMBER)

        # Wait for device to close #

```

```

mDLL.wait(1000)

# Quit the library #
mDLL.quit()

try:
    sys.exit(0)
except SystemExit:
    os._exit(0)

```

6.5.2 C++

```

#include <iostream>

#include "doric_system_wrapper.h"

int main()
{
    /* Create a Rotary Joint settings*/
    Doric::RotaryJoint::Settings* rj_settings = new Doric::RotaryJoint::Settings();

    /* Configure a Rotary Joint for normal speed */
    rj_settings->samplingRate = Doric::RotaryJoint::RotaryJoint::kFreq_120Hz;
    rj_settings->motorSpeed = Doric::RotaryJoint::MotorSpeedFactor::kSPEED_FACTOR_NORMAL;

    /* Initialize communication with Doric devices */
    Doric::System::init(true);

    /* Wait for initialization to complete*/
    Doric::System::wait(5000);

    /* List available(s) device(s) */
    Doric::System::available_devices_with_ports();

    /* Rotary Joint USB port */
    int RJ_port = 10;

    /* Open a device. In this case, a Rotary Joint is connected on port 10 */
    Doric::System::open_device(RJ_port);

    /* Wait for the device to be initialize */
    Doric::System::wait(7000);

    /* Send the settings created previously to device on port #10 */
    Doric::RotaryJoint::rotary_joint_send_settings(RJ_port, rj_settings);

    /* Wait for the settings to be initialize */
    Doric::System::wait(1000);

    /* Start the Rotary Joint (connected on port#10) */
    Doric::RotaryJoint::rotary_joint_send_motor_power_on(RJ_port);

    /* Set maximum duty cycle to the Rotary Joint */
    Doric::RotaryJoint::rotary_joint_send_manual_control_duty_cycle(RJ_port, 100);

    /* Activate manual control */
    Doric::RotaryJoint::rotary_joint_send_manual_control_activated(RJ_port);
}

```

```

/* Set the manual control (to fully manual) */
Doric::RotaryJoint::rotary_joint_send_manual_control_mode(RJ_port, Doric::RotaryJoint::MotorMode::Manual);

/* Make it turn clockwise */
Doric::RotaryJoint::rotary_joint_send_manual_control_direction(RJ_port, Doric::RotaryJoint::MotorDirection::Clockwise);

/* Let it run 10 sec */
Doric::System::wait(10000);

/* Make it turn counter-clockwise */
Doric::RotaryJoint::rotary_joint_send_manual_control_direction(RJ_port, Doric::RotaryJoint::MotorDirection::CounterClockwise);

/* Let it run 10 sec */
Doric::System::wait(10000);

/* Stop the Rotary Joint (connected on port#10) */
Doric::RotaryJoint::rotary_joint_send_motor_power_on(RJ_port);

/* Close the Rotary Joint */
Doric::System::close_device(RJ_port);

/* Wait for device to close */
Doric::System::wait(1000);

Doric::System::quit();
}

```

6.5.3 Matlab/Octave

In progress (Matlab code here)

License(s)

- **Qt**
This software uses the Qt development tools library under the [Qt License](https://www.qt.io/)
<https://www.qt.io/>
- **OpenCV**
This software uses the OpenCV library under the [Apache 2 License](https://opencv.org/)
<https://opencv.org/>
- **FFmpeg**
This software uses libraries from the FFmpeg project under the [LGPLv2.1 License](https://ffmpeg.org/)
<https://ffmpeg.org/>
- **QCustomPlot**
This software uses the QCustomPlot project under the [GNU GPLv3 or later License](https://www.qcustomplot.com/)
<https://www.qcustomplot.com/>
- **HDF5**
This software uses the HDF5 library under the specific [HDF Group HDF5 License](https://www.hdfgroup.org/)
<https://www.hdfgroup.org/>
- **Font Awesome**
This software uses the Font Awesome library under the [CC BY 4.0 License \(Icons\)](https://fontawesome.com/), [SIL OFL 1.1 License \(Fonts\)](https://fontawesome.com/) & [MIT License \(Code\)](https://fontawesome.com/)
<https://fontawesome.com/>

Support

8.1 Contact us

For any questions, comments or issues, do not hesitate to contact us by:

Phone 1-418-877-5600

Web doriclenses.com/contact

Email sales@doriclenses.com

The Doric logo consists of the word "doric" in a lowercase, sans-serif font. The letter "o" is stylized with a white circle inside, creating a lens-like effect.

© 2024 DORIC LENSES INC

357 rue Franquet - Quebec, (Quebec)

G1P 4N7, Canada

Phone: 1-418-877-5600 - Fax: 1-418-877-1008

www.doriclenses.com